Dubins Path: the Mathematics of `dubins_path.py`

November 2026

This note states every formula that `dubins_path.py` (in `Dubins_TSP_New_Implementation_claude/`) computes, in the notation of Section 0.13 of `Dubins_TSP.pdf`. Beside each symbol is the name it has in the code, and each section names the functions it describes, so every formula can be found in the program. All closed forms below were checked against the code on 20 000 random pairs of poses ($R = 5$ m, points in a 40 m square), which gave 74 236 CSC and 18 450 CCC paths: they agree to within 10^{-13} m.

1 Vehicle Model and Notation

Given two positions

$$(x_1, y_1, \psi_1), \qquad (x_2, y_2, \psi_2),$$

we are to find a Dubins path of minimum turn radius R from the first to the second. The mobile sink moves forward only, at constant speed v:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \end{bmatrix} = v \begin{bmatrix} \cos\psi \\ \sin\psi \\ \frac{\alpha}{R} \end{bmatrix}, \qquad \alpha \in [-1, 1]. \tag{1}$$

This is eq. (21) of the paper, with the heading written ψ as in Section 0.13. Headings are in radians, measured anticlockwise from the x axis. A Dubins path uses only the three extreme controls: $\alpha = 1$ (a left arc of radius R), $\alpha = -1$ (a right arc of radius R) and $\alpha = 0$ (a straight line). The direction of an arc is

$$\sigma = \begin{cases} +1 & \text{left turn (anticlockwise)}, \\ -1 & \text{right turn (clockwise)}, \end{cases} \tag{2}$$

which is `SIGMA = {"L": 1, "R": -1}` in the code. A path has three parts, and its turn type (`turnType`) is one of the six words

$$\text{LSL}, \quad \text{RSR}, \quad \text{LSR}, \quad \text{RSL}, \quad \text{RLR}, \quad \text{LRL}$$

(`TURN_TYPES`), where L is a left arc, R a right arc and S a straight line. σ_1 is the direction of the first letter and σ_2 that of the last. In a CSC path the middle part is straight. In a CCC path $\sigma_1 = \sigma_2$ and the middle arc turns the other way, with direction $-\sigma_1$.

Table 1 lists the symbols. A centre of a circle is always written with a digit, (x_{c1}, y_{c1}), (x_{c2}, y_{c2}) or (x_{c3}, y_{c3}), so (x_{ci}, y_{ci}) always means the cut-in point.

2 Basic Geometry

Centre of a turning circle (`returnCircleCenter`). The circle of direction σ that passes through (x, y) with heading ψ has its centre a distance R to the left ($\sigma = 1$) or to the right ($\sigma = -1$) of the heading:

$$(x_c, y_c) = \Big(x + R\cos\big(\psi + \tfrac{\pi}{2}\sigma\big),\ y + R\sin\big(\psi + \tfrac{\pi}{2}\sigma\big) \Big). \tag{3}$$

The first circle uses $(x_1, y_1, \psi_1, \sigma_1)$ and gives (x_{c1}, y_{c1}). The last circle uses $(x_2, y_2, \psi_2, \sigma_2)$ and gives (x_{c2}, y_{c2}).

Distance and direction between the centres (`distance`, `atan2`).

$$S_{\tan} = \sqrt{(x_{c2} - x_{c1})^2 + (y_{c2} - y_{c1})^2}, \tag{4}$$

$$\psi_L = \operatorname{atan2}(y_{c2} - y_{c1},\ x_{c2} - x_{c1}). \tag{5}$$

atan2 gives the angle in $(-\pi, \pi]$ in the correct quadrant and divides by nothing, so it is defined even when $x_{c2} = x_{c1}$.

Table 1: Symbols and their names in `dubins_path.py`.

Symbol	Code	Meaning
x_1, y_1, ψ_1	`x1, y1, psi1`	start point and start heading
x_2, y_2, ψ_2	`x2, y2, psi2`	end point and end heading
R	`R`	minimum turning radius (5 m with the defaults of `config.py`)
σ_1, σ_2	`sigma1, sigma2`	direction of the first and the last arc, eq. (2)
$\hat{u}_1, u_2, \hat{u}_3$	`u_vec1, u_vec2, u_vec3`	vectors of the turn-type rule
cp_1, cp_2	`cp1, cp2`	cross products of the turn-type rule
$(x_{c1}, y_{c1}), (x_{c2}, y_{c2})$	`xc1, yc1, xc2, yc2`	centres of the first and the last circle
(x_{c3}, y_{c3})	`xc3, yc3`	centre of the middle circle of a CCC path
$S_{\tan}$	`Stan`	distance between (x_{c1}, y_{c1}) and (x_{c2}, y_{c2})
ψ_L	`psiL`	direction of the line from the first to the last centre
ψ_D	`psiD`	heading of the straight part (the cut-out angle ψ_d of Section 0.13)
β	`cutOutAngle`	angle between the straight part and the line of centres (LSR, RSL)
θ	`theta`	angle at (x_{c1}, y_{c1}) between the lines to the other two centres (CCC)
ψ_{co}, ψ_{ci}	`psiCO, psiCI`	headings at the cut-out and the cut-in point (CCC)
(x_{co}, y_{co})	`xco, yco`	cut-out point, where the first arc ends
(x_{ci}, y_{ci})	`xci, yci`	cut-in point, where the last arc starts
$\hat{u}, d_{co,ci}$	`u_hat, dist`	direction and length of the straight part
S_1, S_3, S_2	`S1, S3, S2`	lengths of the first, the middle (CCC) and the last arc
S_{path}	`pathDistance`	total length of the path
ψ_T	`psiT`	heading at the end of the part computed so far
Δs	`step`	spacing of the points along the path (`p["ds"]` $= 0.25$ m)
$x_{\text{trajectory}}, y_{\text{trajectory}}$	`xtrajectory, ytrajectory`	the points of the path
$s_{\text{trajectory}}$	`strajectory`	distance along the path of each point

Turning angle (`turnAngle`). On a circle of direction σ the heading can only change in the direction σ. To go from heading ψ_a to heading ψ_b the sink therefore turns through

$$\Delta_\sigma(\psi_a, \psi_b) = \big[\sigma(\psi_b - \psi_a)\big] \bmod 2\pi, \qquad 0 \le \Delta_\sigma(\psi_a, \psi_b) < 2\pi, \tag{6}$$

and drives an arc of length $R\,\Delta_\sigma(\psi_a, \psi_b)$. Here $a \bmod 2\pi = a - 2\pi\lfloor a/2\pi \rfloor$ is never negative, so no check is needed for a heading passing π or $-\pi$. To stop a rounding error from turning "no turn" into a full circle, the code sets

$$\Delta_\sigma(\psi_a, \psi_b) = 0 \quad \text{if} \quad \Delta_\sigma(\psi_a, \psi_b) > 2\pi - 10^{-6}. \tag{7}$$

Headings can be off by about 10^{-7} rad when $S_{\tan}$ is almost exactly $2R$, because $\sin^{-1}$ is very sensitive near 1.

Driving along an arc (`pointsOnArc`). A sink that starts at (x, y) with heading ψ and drives s metres on the circle of direction σ, whose centre (x_c, y_c) is given by eq. (3), is at

$$\begin{bmatrix} x(s) \\ y(s) \end{bmatrix} = \begin{bmatrix} \cos\alpha & -\sigma\sin\alpha \\ \sigma\sin\alpha & \cos\alpha \end{bmatrix} \begin{bmatrix} x - x_c \\ y - y_c \end{bmatrix} + \begin{bmatrix} x_c \\ y_c \end{bmatrix}, \qquad \alpha = \frac{s}{R}, \tag{8}$$

with heading

$$\psi(s) = \psi + \sigma\,\frac{s}{R}. \tag{9}$$

Eq. (8) is the rotation matrix of Section 0.13.1 with the exact angle s/R in place of the step $i\,\Delta\psi$.

3 The Turn-Type Rule of Section 0.13

These formulas are in `findTurnType`, `findCSCTurnType` and `findCCCTurnType`. The path uses them only when `dubins_method` is `"pdf"` in `config.py`. Define

$$\hat{u}_1 = \begin{bmatrix}\cos\psi_1\\ \sin\psi_1\end{bmatrix}, \qquad u_2 = \begin{bmatrix}x_2 - x_1\\ y_2 - y_1\end{bmatrix}, \qquad \hat{u}_3 = \begin{bmatrix}\cos\psi_2\\ \sin\psi_2\end{bmatrix}. \tag{10}$$

Only the signs of the cross products are used, so u_2 is not divided by its length; this also avoids dividing by zero when the two points coincide. The cross products are

$$\begin{aligned} cp_1 &= \hat{u}_1 \times u_2 = \cos\psi_1\,(y_2 - y_1) - \sin\psi_1\,(x_2 - x_1),\\ cp_2 &= u_2 \times \hat{u}_3 = (x_2 - x_1)\sin\psi_2 - (y_2 - y_1)\cos\psi_2. \end{aligned} \tag{11}$$

$cp_1 > 0$ when the end point lies to the left of the start heading, and $cp_2 > 0$ when the end heading points to the left of u_2. The CSC turn type is determined as follows:

$$\begin{cases} cp_1 \le 0,\ cp_2 \ge 0 & \Rightarrow \text{turn type} = \text{RSL}, \quad \sigma_1 = -1,\ \sigma_2 = 1,\\ cp_1 \le 0,\ cp_2 < 0 & \Rightarrow \text{turn type} = \text{RSR}, \quad \sigma_1 = -1,\ \sigma_2 = -1,\\ cp_1 > 0,\ cp_2 < 0 & \Rightarrow \text{turn type} = \text{LSR}, \quad \sigma_1 = 1,\ \sigma_2 = -1,\\ cp_1 > 0,\ cp_2 \ge 0 & \Rightarrow \text{turn type} = \text{LSL}, \quad \sigma_1 = 1,\ \sigma_2 = 1. \end{cases} \tag{12}$$

The case $cp_2 = 0$, where the end heading lies along u_2, is treated like $cp_2 > 0$. Then the centres follow from eq. (3) with these σ_1, σ_2, and $S_{\tan}$ from eq. (4). Only RSR and LSL can become CCC curves:

$$\text{if } 2R \le S_{\tan} < 3R: \qquad \text{RSR} \Rightarrow \text{RLR}\ (\sigma_1 = \sigma_2 = -1), \qquad \text{LSL} \Rightarrow \text{LRL}\ (\sigma_1 = \sigma_2 = 1). \tag{13}$$

LSR and RSL stay CSC curves for every $S_{\tan}$. The turn type chosen this way may not exist for the given poses (LSR or RSL with $S_{\tan} < 2R$); Section 5 says what is done then.

4 Lengths of the Three Parts

These formulas (`calculateCSCLengths`, `calculateCCCLengths`) take one turn type and return the lengths of its three parts. Each turn type has its own σ_1 and σ_2, and therefore its own centres, $S_{\tan}$ and ψ_L from eqs. (3)–(5). The functions accept arrays of headings as well as single numbers, so one call gives the lengths for many start and end headings at once.

4.1 CSC Paths

Where the straight part, with heading ψ_D, touches circle k, the centre of that circle lies a distance R from the straight part, on side σ_k. Going from the first centre to the second therefore means going R back to the straight part, along it for its length $d_{co,ci}$, and R out to the second centre:

$$\begin{bmatrix}x_{c2} - x_{c1}\\ y_{c2} - y_{c1}\end{bmatrix} = d_{co,ci}\begin{bmatrix}\cos\psi_D\\ \sin\psi_D\end{bmatrix} + (\sigma_2 - \sigma_1)\,R\begin{bmatrix}-\sin\psi_D\\ \cos\psi_D\end{bmatrix}. \tag{14}$$

RSR and LSL ($\sigma_1 = \sigma_2$). The second term of eq. (14) vanishes, so the straight part is parallel to the line of centres and as long as it:

$$\psi_D = \psi_L, \qquad d_{co,ci} = S_{\tan}. \tag{15}$$

These turn types exist for every $S_{\tan}$. When $S_{\tan} < 10^{-9}$ the two circles are the same and ψ_L has no meaning; the code then sets $\psi_D = \psi_1$, so that $S_1 = 0$, $d_{co,ci} = 0$ and the sink simply keeps turning on the one circle.

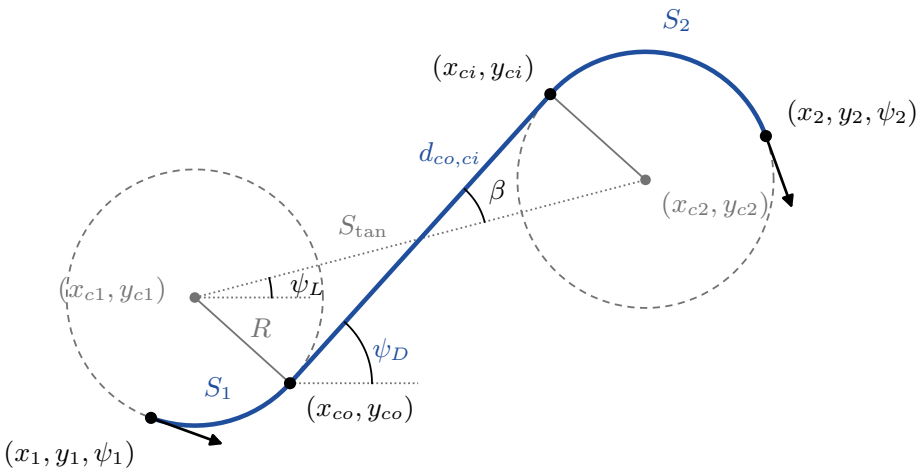


Figure 1: An LSR path, drawn by `dubins_path.py` from $(0, 0, -20°)$ to $(24, 11, -70°)$ with $R = 5\,\text{m}$. The circles' centres are $S_{\tan} = 18.18\,\text{m}$ apart in direction $\psi_L = 14.6°$. The straight part crosses the line of centres at its midpoint, at the angle $\beta = \sin^{-1}(2R/S_{\tan}) = 33.4°$, so its heading is $\psi_D = \psi_L + \beta = 48.0°$. The lengths are $S_1 = 5.93$, $d_{co,ci} = 15.18$ and $S_2 = 10.30\,\text{m}$. Here LSR is both the shortest path and the choice of the rule of Section 0.13.

LSR and RSL ($\sigma_2 = -\sigma_1$). The two terms of eq. (14) are at right angles, so $S_{\tan}^2 = d_{co,ci}^2 + 4R^2$, and the line of centres makes the angle β with the straight part (Figure 1):

$$\beta = \sin^{-1}\left(\frac{2R}{S_{\tan}}\right), \qquad \psi_D = \psi_L + \sigma_1\beta, \qquad d_{co,ci} = \sqrt{S_{\tan}^2 - 4R^2}. \tag{16}$$

That is, $\psi_D = \psi_L + \sin^{-1}(2R/S_{\tan})$ for LSR and $\psi_D = \psi_L - \sin^{-1}(2R/S_{\tan})$ for RSL. These turn types exist only if

$$S_{\tan} \geq 2R, \tag{17}$$

because closer than that the two circles overlap and no straight line can touch both on opposite sides.

Arc lengths. By eq. (6), the first arc turns from ψ_1 to ψ_D and the last arc from ψ_D to ψ_2:

$$S_1 = R\,\Delta_{\sigma_1}(\psi_1, \psi_D) = R\left[\sigma_1(\psi_D - \psi_1) \bmod 2\pi\right], \qquad S_2 = R\,\Delta_{\sigma_2}(\psi_D, \psi_2) = R\left[\sigma_2(\psi_2 - \psi_D) \bmod 2\pi\right]. \tag{18}$$

The total path distance is

$$\boxed{S_{\text{path}} = S_1 + d_{co,ci} + S_2.} \tag{19}$$

If the turn type does not exist, the code returns $S_1 = d_{co,ci} = S_2 = \infty$, so it is never the shortest.

4.2 CCC Paths

Here $\sigma_1 = \sigma_2$, and the middle circle, of direction $-\sigma_1$, touches both outer circles, so its centre is $2R$ from each of theirs. Such a circle exists only if

$$S_{\tan} \leq 4R. \tag{20}$$

The three centres form a triangle with sides $2R$, $2R$ and $S_{\tan}$ (Figure 2). Its angle at (x_{c1}, y_{c1}) is

$$\theta = \cos^{-1}\left(\frac{S_{\tan}}{4R}\right), \qquad 0 \leq \theta \leq \frac{\pi}{2}, \tag{21}$$

and the centre of the middle circle is

$$x_{c3} = x_{c1} + 2R\cos(\psi_L + \sigma_1\theta), \qquad y_{c3} = y_{c1} + 2R\sin(\psi_L + \sigma_1\theta). \tag{22}$$

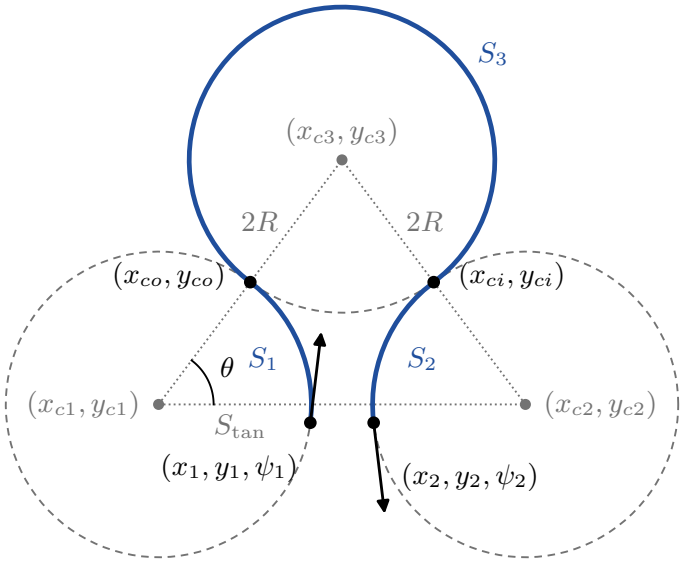


Figure 2: An LRL path, drawn by `dubins_path.py` with $R = 5\,\mathrm{m}$. The outer centres are $S_{\tan} = 12\,\mathrm{m}$ apart, so $\theta = \cos^{-1}(12/20) = 53.13^\circ$. The lengths are $S_1 = S_2 = 5.24\,\mathrm{m}$ and $S_3 = R(\pi + 2\theta) = 24.98\,\mathrm{m}$. For these poses LRL is the shortest of the six turn types.

Two circles touch halfway between their centres, and there the sink's heading is at a right angle to the line joining the centres. This gives the headings at the cut-out and the cut-in point:

$$\psi_{co} = \psi_L + \sigma_1\Big(\theta + \frac{\pi}{2}\Big), \qquad \psi_{ci} = \psi_L - \sigma_1\Big(\theta + \frac{\pi}{2}\Big). \tag{23}$$

By eq. (6), the arc lengths are

$$\begin{aligned} S_1 &= R\,\Delta_{\sigma_1}(\psi_1, \psi_{co}) = R\left[\sigma_1(\psi_{co} - \psi_1) \bmod 2\pi\right], \\ S_3 &= R\,\Delta_{-\sigma_1}(\psi_{co}, \psi_{ci}) = R\left[-\sigma_1(\psi_{ci} - \psi_{co}) \bmod 2\pi\right] = R\left[(\pi + 2\theta) \bmod 2\pi\right], \\ S_2 &= R\,\Delta_{\sigma_2}(\psi_{ci}, \psi_2) = R\left[\sigma_2(\psi_2 - \psi_{ci}) \bmod 2\pi\right], \end{aligned} \tag{24}$$

and the total path distance is

$$\boxed{S_{\mathrm{path}} = S_1 + S_3 + S_2.} \tag{25}$$

Because $0 \le \theta \le \pi/2$, the middle arc is $S_3 = R(\pi + 2\theta)$, at least half a circle, whenever $S_{\tan} > 0$. When $S_{\tan} = 0$ the two outer circles coincide, $\theta = \pi/2$, and the code gives $S_3 = 0$ instead of $2\pi R$: the path then stays on the one circle. Of the two circles that touch both outer circles, the code always uses the one on side σ_1 of the line of centres, whose arc is the longer one, $S_3 \ge \pi R$. Only that one can belong to a shortest path [1]. If $S_{\tan} > 4R$, the code returns $S_1 = S_3 = S_2 = \infty$.

4.3 When Each Turn Type Exists

Table 2: Existence of the six turn types. $S_{\tan}$ is computed with the centres of that turn type.

Turn type	Middle part	Exists when
LSL, RSR	straight, $d_{co,ci} = S_{\tan}$	always
LSR, RSL	straight, $d_{co,ci} = \sqrt{S_{\tan}^2 - 4R^2}$	$S_{\tan} \ge 2R$
RLR, LRL	arc, $S_3 = R(\pi + 2\theta)$	$S_{\tan} \le 4R$

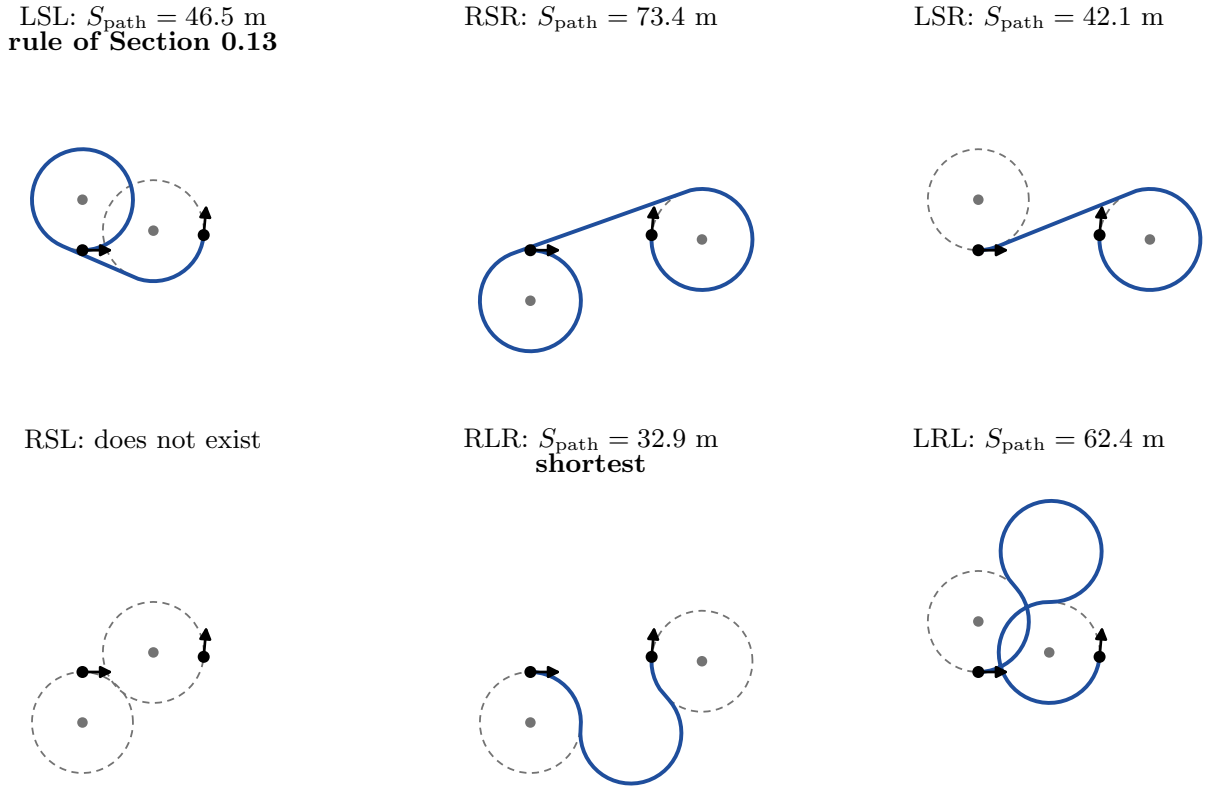


Figure 3: The six turn types from $(0, 0, 0°)$ to $(12, 1.5, 85°)$ with $R = 5$ m, drawn by `dubins_path.py`. RSL does not exist: its circles are $S_{\text{tan}} = 9.87$ m apart, less than $2R$. Here $cp_1 = 1.5 > 0$ and $cp_2 = 11.82 > 0$, so the rule of Section 0.13 picks LSL; its circles are 7.66 m apart, below $2R$, so it stays LSL. LSL is 46.5 m long, 41 % longer than the shortest path, RLR (32.9 m).

5 Choosing the Path

For a turn type T, let $S_{\text{path}}(T)$ be its total length from eq. (19) or eq. (25), with $S_{\text{path}}(T) = \infty$ if T does not exist. The function `findDubinsPath` chooses the turn type in one of two ways, set by `p["dubins_method"]` in `config.py`.

`"shortest"` (the default). Compute all six and keep the shortest:

$$T^* = \arg\min_T S_{\text{path}}(T), \qquad T \in \{\text{LSL}, \text{RSR}, \text{LSR}, \text{RSL}, \text{RLR}, \text{LRL}\}. \tag{26}$$

A tie goes to the turn type that comes first in the order LSL, RSR, LSR, RSL, RLR, LRL. Dubins proved that a shortest path with turn radius at least R is of type CSC or CCC, with the middle arc of a CCC path longer than πR [1]. For each turn type the formulas of Section 4 give the only such candidate (with every arc shorter than a full circle), so T^* is the shortest path.

`"pdf".` Take the turn type T_{rule} of Section 3, unless it does not exist:

$$T = \begin{cases} T_{\text{rule}} & \text{if } S_{\text{path}}(T_{\text{rule}}) < \infty, \\ T^* & \text{otherwise.} \end{cases} \tag{27}$$

Section 0.13 does not say what to do when T_{rule} does not exist; the code uses the shortest turn type that does. The rule does not always give the shortest path (Figure 3): the signs of cp_1 and cp_2 only say on which

side the end point and the end heading lie, and a CCC curve, which can exist up to $S_{\tan} = 4R$ and can be the shortest above $3R$, is tried only for $2R \le S_{\tan} < 3R$.

Lengths only (`returnPathDistances`). The heading search of `trajectory.py` needs only the length, for many headings at once. With start headings $\psi_1^{(i)}$ and end headings $\psi_2^{(j)}$ it computes the table

$$L_{ij} = S_{\text{path}}\big(T_{ij}\big) \quad \text{for the poses } \big(x_1, y_1, \psi_1^{(i)}\big),\ \big(x_2, y_2, \psi_2^{(j)}\big), \tag{28}$$

where T_{ij} is chosen by eq. (26) or eq. (27), as in `findDubinsPath`.

6 The Points of the Path

These formulas are in `calculateCSCTrajectory`, `calculateCCCTrajectory` and `returnDubinsPath`.

Spacing. The points are placed at the distances along the path

$$s_j = j\,\Delta s, \quad j = 0, 1, \ldots, J-1, \qquad J = \left\lceil \frac{S_{\text{path}}}{\Delta s} \right\rceil, \qquad \text{and} \qquad s_J = S_{\text{path}}, \tag{29}$$

so consecutive points are Δs apart along the path on every part, except that the end point is always included. The default is $\Delta s = 0.25\,\text{m}$. A part that starts at distance a and has length ℓ gets the points with $a \le s_j < a + \ell$, at the distance $s_j - a$ from its own start.

6.1 CSC Trajectory

First circular arc: Cut-out point. For $0 \le s < S_1$ the point is given by eq. (8) from (x_1, y_1, ψ_1) with direction σ_1. At $s = S_1$, with $\alpha_1 = S_1/R$, it reaches the cut-out point

$$\boxed{\begin{aligned} x_{co} &= \cos\alpha_1\,(x_1 - x_{c1}) - \sigma_1 \sin\alpha_1\,(y_1 - y_{c1}) + x_{c1}, \\ y_{co} &= \sigma_1 \sin\alpha_1\,(x_1 - x_{c1}) + \cos\alpha_1\,(y_1 - y_{c1}) + y_{c1}, \end{aligned}} \tag{30}$$

with heading, by eq. (9),

$$\psi_D = \psi_1 + \sigma_1 \frac{S_1}{R}, \tag{31}$$

which is the ψ_D of eq. (15) or eq. (16) up to a multiple of 2π.

Straight trajectory. The unit vector of the straight part is

$$\hat{u} = \begin{bmatrix} \cos\psi_D \\ \sin\psi_D \end{bmatrix}, \tag{32}$$

and for $S_1 \le s < S_1 + d_{co,ci}$ the point is

$$\begin{bmatrix} x(s) \\ y(s) \end{bmatrix} = \begin{bmatrix} x_{co} \\ y_{co} \end{bmatrix} + (s - S_1)\,\hat{u}. \tag{33}$$

The straight part ends at the cut-in point

$$\boxed{\begin{bmatrix} x_{ci} \\ y_{ci} \end{bmatrix} = \begin{bmatrix} x_{co} \\ y_{co} \end{bmatrix} + d_{co,ci}\,\hat{u}.} \tag{34}$$

Second circular arc. For $S_1 + d_{co,ci} \le s < S_{\text{path}}$ the point is given by eq. (8) from (x_{ci}, y_{ci}, ψ_D) with direction σ_2 and distance $s - S_1 - d_{co,ci}$; by eq. (3) the centre of this circle is (x_{c2}, y_{c2}). The end point, at distance S_2 on this arc, is (x_2, y_2), and the final heading is

$$\psi_T = \psi_D + \sigma_2 \frac{S_2}{R} \equiv \psi_2 \pmod{2\pi}. \tag{35}$$

Comparison with Section 0.13. Section 0.13 finds the cut-in point by turning the end point back through $\alpha_2 = S_2/R$ about (x_{c2}, y_{c2}),

$$\begin{aligned} x_{ci} &= \cos\alpha_2\,(x_2 - x_{c2}) + \sigma_2 \sin\alpha_2\,(y_2 - y_{c2}) + x_{c2}, \\ y_{ci} &= -\sigma_2 \sin\alpha_2\,(x_2 - x_{c2}) + \cos\alpha_2\,(y_2 - y_{c2}) + y_{c2}, \end{aligned} \tag{36}$$

and takes $\hat{u}$ as the unit vector from the cut-out to the cut-in point. The code instead goes forward with eqs. (32) and (34), so each part starts exactly where the previous one ended. Both give the same point: the two differ by less than 10^{-13} m in the check described at the start.

6.2 CCC Trajectory

First circular arc. As in the CSC case: eq. (8) from (x_1, y_1, ψ_1) with direction σ_1 for $0 \le s < S_1$, reaching the cut-out point of eq. (30) with S_1 from eq. (24), and the heading $\psi_T = \psi_1 + \sigma_1 S_1/R \equiv \psi_{co}$.

Middle circular arc. For $S_1 \le s < S_1 + S_3$ the point is given by eq. (8) from (x_{co}, y_{co}, ψ_T) with direction $-\sigma_1$ and distance $s - S_1$. By eqs. (3) and (23) the centre of this circle is

$$x_{co} + R\cos\Big(\psi_{co} - \sigma_1\frac{\pi}{2}\Big) = x_{c1} + 2R\cos(\psi_L + \sigma_1\theta), \qquad y_{co} + R\sin\Big(\psi_{co} - \sigma_1\frac{\pi}{2}\Big) = y_{c1} + 2R\sin(\psi_L + \sigma_1\theta),$$

which is the centre (x_{c3}, y_{c3}) of eq. (22). At distance S_3 it reaches the cut-in point, with heading $\psi_T \leftarrow \psi_T - \sigma_1 S_3/R \equiv \psi_{ci}$.

Last circular arc. For $S_1 + S_3 \le s < S_{\text{path}}$ the point is given by eq. (8) from (x_{ci}, y_{ci}, ψ_T) with direction σ_2 and distance $s - S_1 - S_3$. It ends at (x_2, y_2) with heading $\psi_T \leftarrow \psi_T + \sigma_2 S_2/R \equiv \psi_2 \pmod{2\pi}$.

Closed forms. The cut-out and cut-in points computed this way are the touching points of the circles, halfway between their centres, as written in Section 0.13.2:

$$\begin{aligned} x_{co} &= x_{c1} + R\cos(\psi_L + \sigma_1\theta), \quad y_{co} = y_{c1} + R\sin(\psi_L + \sigma_1\theta), \\ x_{ci} &= x_{c2} - R\cos(\psi_L - \sigma_1\theta), \quad y_{ci} = y_{c2} - R\sin(\psi_L - \sigma_1\theta). \end{aligned} \tag{37}$$

6.3 Output

`returnDubinsPath` returns

$$\big(x_{\text{trajectory}},\ y_{\text{trajectory}},\ S_{\text{path}},\ s_{\text{trajectory}}\big), \tag{38}$$

where

$$x_{\text{trajectory}} = \big(x(s_0), \ldots, x(s_J)\big), \qquad y_{\text{trajectory}} = \big(y(s_0), \ldots, y(s_J)\big), \qquad s_{\text{trajectory}} = (s_0, \ldots, s_J).$$

At the constant speed of eq. (1), the sink reaches point j at time s_j/v after the start of the path. These times are what `trajectory.py` uses to place the sink on the shared timeline.

7 What the Construction Guarantees

- **No corners.** Each part starts at the point and with the heading at which the previous part ended (eqs. (8), (9), (31) and (34)), so the position and the heading are continuous along the whole path.
- **Turning radius.** Every arc has radius exactly R and the straight part has none, so the path never turns tighter than R. Every part has a length of at least 0, by eq. (6).
- **End pose.** The last point is (x_2, y_2) and the final heading is ψ_2 up to a multiple of 2π, apart from rounding: at most 8×10^{-14} m and 4×10^{-15} rad in the check described at the start. The test `test_dubins_every_turn_type_reaches_the_target_smoothly` checks this for every turn type.
- **Shortest.** With `"shortest"` the path is the shortest path with turning radius at least R (Section 5). With `"pdf"` it can be longer (Figure 3).

8 Numerical Safeguards

Table 3 lists every tolerance and clamp in the code. None of them changes a result by more than rounding; they stop a rounding error from deciding whether a path exists, or from producing a full circle or an invalid $\sin^{-1}$, $\cos^{-1}$ or square root.

Table 3: Tolerances and clamps in `dubins_path.py`.

Where	What the code does	Why
`turnAngle`	$\Delta > 2\pi - 10^{-6} \Rightarrow \Delta = 0$	"no turn" must not become a full circle
RSR, LSL	$S_{\tan} < 10^{-9} \Rightarrow \psi_D = \psi_1$	ψ_L means nothing when the circles coincide
LSR, RSL	exists if $S_{\tan} \geq 2R - 10^{-9}$	circles that only just touch count
LSR, RSL	$\beta = \sin^{-1}\big(\min\{2R/\max(S_{\tan}, 10^{-9}),\, 1\}\big)$	no division by zero; $\sin^{-1}$ defined
LSR, RSL	$d_{co,ci} = \sqrt{\max(S_{\tan}^2 - 4R^2,\, 0)}$	no square root of a negative number
RLR, LRL	exists if $S_{\tan} \leq 4R + 10^{-9}$	circles that only just touch count
RLR, LRL	$\theta = \cos^{-1}\big(\min\{S_{\tan}/4R,\, 1\}\big)$	$\cos^{-1}$ defined
all	a turn type that does not exist has length ∞	it is never the shortest

9 Relation to Section 0.13 of the Paper

Table 4 compares the original text of Section 0.13 with the code. The blue additions marked in Section 0.13 of `Dubins_TSP.pdf` state the formulas of the right-hand column.

Table 4: Section 0.13 (original text) and `dubins_path.py`.

Topic	Section 0.13	`dubins_path.py`
Choice of turn type	rule of eq. (12) only	rule (`"pdf"`) or shortest of six (`"shortest"`, default), eqs. (26)–(27)
$cp_2 = 0$	not covered	treated like $cp_2 > 0$
u_2	unit vector	not divided by its length
Which types become CCC	"if $2R \leq S_{\tan} < 3R$" for every type	only RSR and LSL, eq. (13)
ψ_L	$(y_{c2} - y_{c1})/(x_{c2} - x_{c1})$	atan2, eq. (5)
ψ_D for LSR, RSL	$\psi_L \pm \tan^{-1}(2R/S_{\tan})$	$\psi_L \pm \sin^{-1}(2R/S_{\tan})$, eq. (16)
Arcs	steps of $\Delta\psi = 0.05$ until $\lvert\psi_T - \psi_D\rvert \leq 0.1$	exact angle, eqs. (6), (18), (24)
Turn type that does not exist	not covered	shortest turn type that exists
CCC geometry	not given	eqs. (21)–(25), (37)
Spacing of points	3 on the straight part only	Δs on every part, eq. (29)

Reference

[1] L. E. Dubins, "On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents," American Journal of Mathematics, vol. 79, no. 3, pp. 497–516, 1957.